Tema 07

Preprocesador y Compilación Separada

Programación en Lenguaje C

Ingeniería de Sistemas y Automática - Universidad de Oviedo

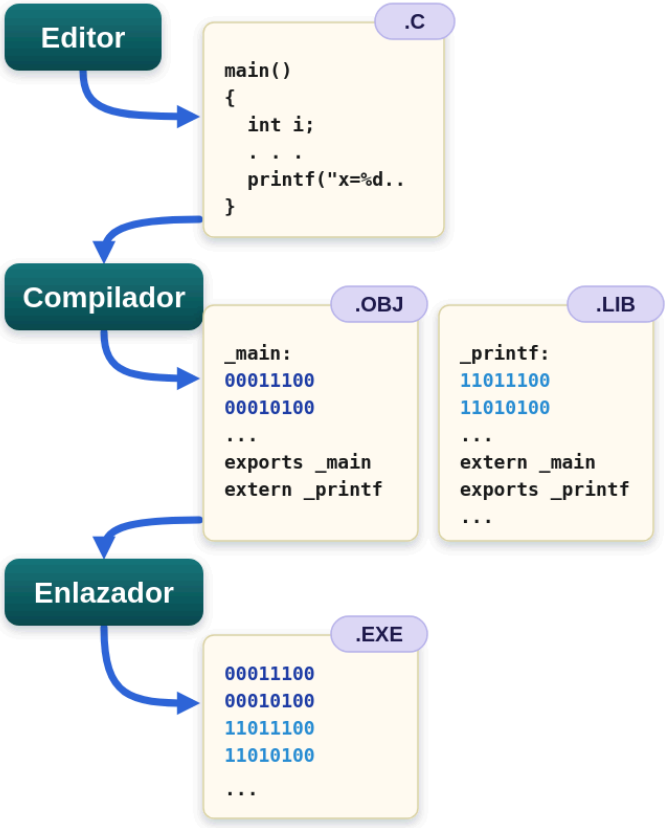
Índice del Tema

1. Del código fuente al código máquina (Editor, Compilador, Enlazador, Cargador)
2. Concepto y ventajas de la Compilación Separada
3. Directivas del Preprocesador (`#include`, `#define`, `#pragma`)
4. Archivos fuente (`.c`) vs. Archivos de cabecera (`.h`)
5. Gestión de variables globales entre módulos (`extern`)
6. Macros con parámetros y prevención de efectos secundarios
7. Compilación Condicional e Inclusión Múltiple (*Include Guards*)

Alto nivel → código máquina

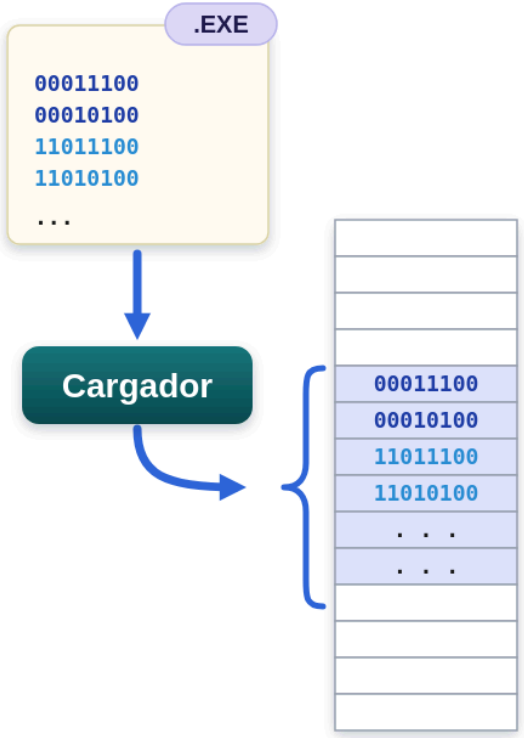
Pasos en la obtención de código máquina:

- **Escribir código fuente**
  (programa editor)
- **Compilar código fuente**
  (programa compilador)
- **Enlazar código objeto y librerías**
  (programa enlazador)



Ejecución del programa

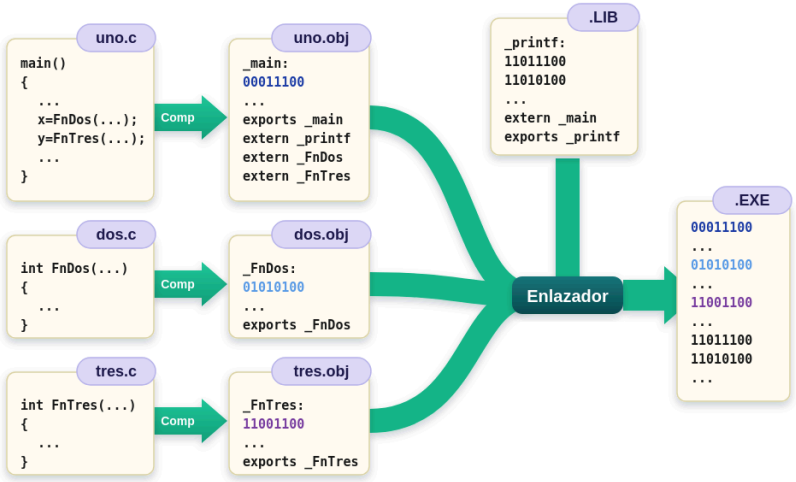
- **Ejecución de un programa:**
  - El programa **Cargador (Loader)** vuelca en memoria el archivo ejecutable.
  - El cargador pone en el **PC (Contador de Programa)** la dirección de la 1ª instrucción del programa (el resto se ejecutan secuencialmente).
  - O bien cargar en memoria y ejecutar paso a paso con un **programa depurador**.



Compilación Separada

Permite dividir programas complejos en múltiples ficheros fuente independientes:

- **Modularidad: Cada fichero `.c` se compila de forma aislada generando su correspondiente fichero objeto `.obj`.**
- **Reutilización y Eficiencia: Al modificar un único archivo, solo es necesario re-compilar ese módulo antes de volver a enlazar.**
- **Enlazado final: El Linker une todos los `.obj` y las librerías estáticas (`.lib`).**



El Preprocesador en C

- Se puede indicar ciertas acciones **previas al compilado** mediante directivas de preprocesador.
- Todas las directivas comienzan por el símbolo # y ocupan una sola línea:

`#include`

Inclusión de un archivo de cabecera.

`#define`

Define constantes simbólicas o macros sustituibles por texto.

`#if / #ifdef / #endif`

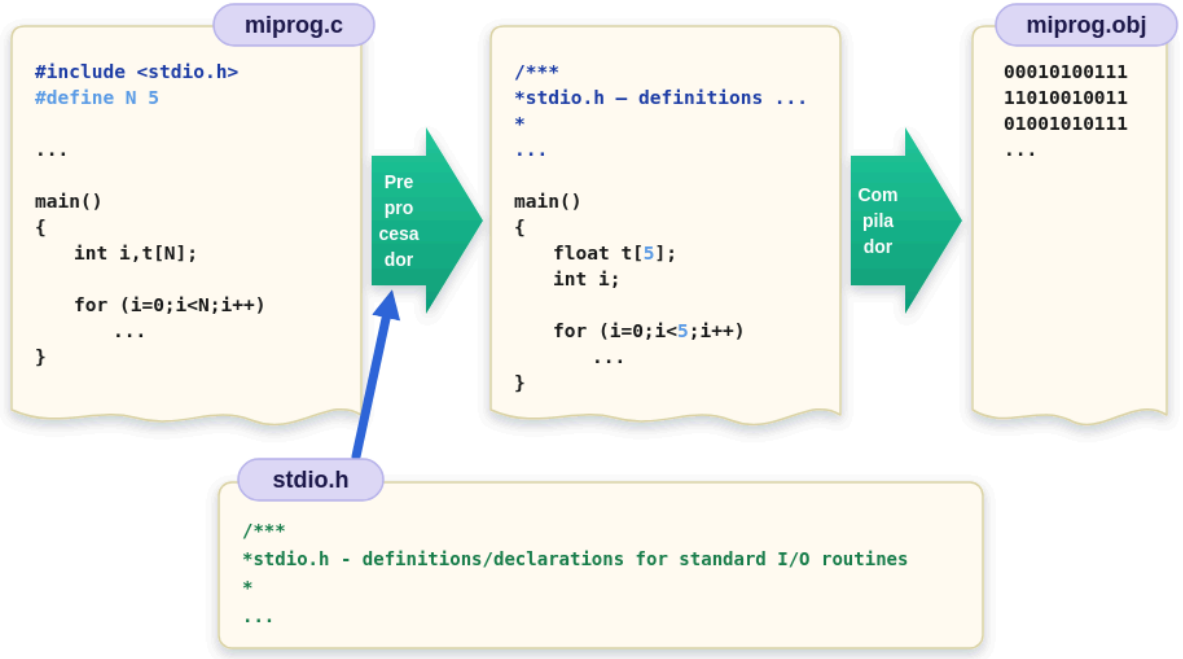
Permiten la compilación condicional del bloque de código.

`#pragma / #error`

Instrucciones específicas al compilador o generación de errores customizados.

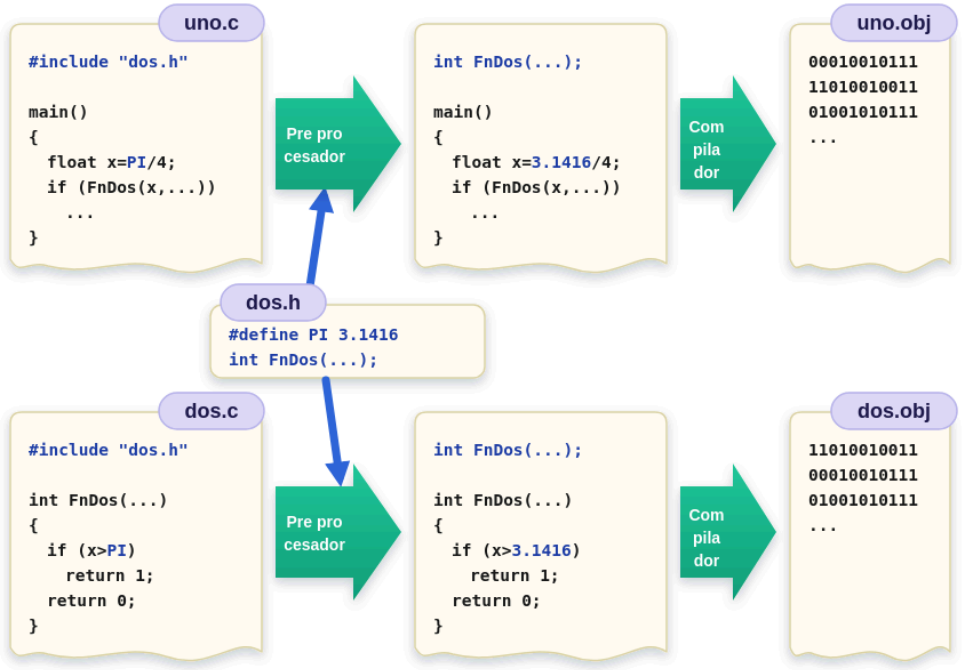
El Preprocesador en C

Las directivas del preprocesador actúan sobre el texto que va a ser compilado, no en tiempo de ejecución.



Preprocesador y compilación separada

Cuando se realiza compilación separada, se utiliza la directiva **#include** para que todos los módulos utilicen las mismas definiciones (prototipos de función, constantes, etc).



Ficheros Fuente (`.c`) vs. Cabeceras (`.h`)

Ficheros Fuente (`.c`)

- Se compilan individualmente para generar archivos `.obj`.
- Contienen el código ejecutable (definición de funciones, sentencias, expresiones).

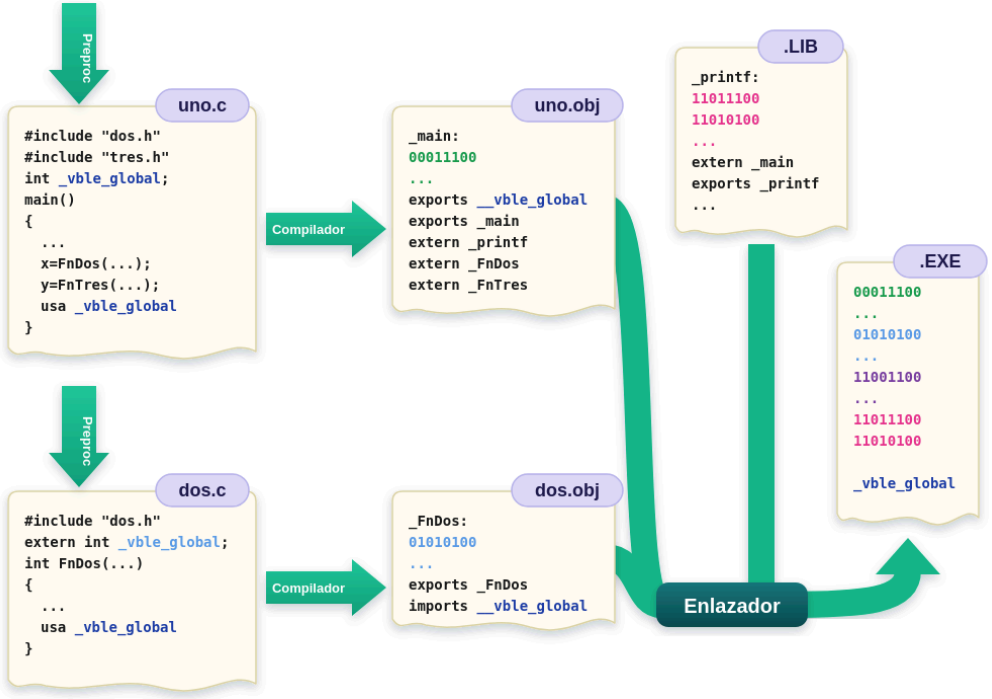
Ficheros de Cabecera (`.h`)

- Nunca se compilan directamente; se incluyen via `#include` en los `.c`.
- Solo contienen **declaraciones** necesarias para los .c que los incluyan
  - Prototipos de función terminados en `;`
  - Constantes e Inclusiones
  - Nuevos tipos (`struct`, `enum`, `typedef`)
  - Otros `#include` necesarios

> Es práctica habitual asociar un fichero de cabecera `.h` a cada módulo `.c`. Este debe contener únicamente las declaraciones imprescindibles (y no más) para utilizar las funciones públicas de dicho módulo desde otros archivos.

Compilación separada

En caso de utilizar variables globales, uno de los módulos declara su almacenamiento, y el resto debe preceder la declaración con extern



El preprocesador

Definición de constantes y macros

- **Sin parámetros:**

```
#define N 50
```

⬅ Todas las apariciones del identificador **N** serán sustituidas por el valor **50** antes de compilar.

- **Con parámetros:**

```
#define MIN(a,b) (a>b) ? a : b
```

Macros con parámetros:

- Aunque su invocación es similar a la de una función, su funcionamiento es diferente. Usar únicamente para código sencillo y repetitivo (ej. `MIN`).
- Evitar problemas en la expansión de macros poniendo paréntesis tanto a los parámetros como a la expresión completa:

```
#define MIN(a,b) ( ((a)>(b)) ? (a) : (b) )
```

El preprocesador

Compilación condicional:

```
#if condición_evaluable_en_tiempo_de_compilación
    // Código a compilar si condición != 0
#else
    // Alternativa opcional
    // Código a compilar si condición == 0
#endif
```

Alternativas a `#if`:

```
#ifdef identif_constante
#ifndef identif_constante
```

Diferenciar entre:

- **`#if... :`** Se produce la comprobación en **tiempo de compilación**. El código que no cumple no forma parte del ejecutable.
- **`if () :`** Se produce la comprobación en **tiempo de ejecución**. Todo el código forma parte del ejecutable.

El preprocesador

- **Ejemplos de compilación condicional:**

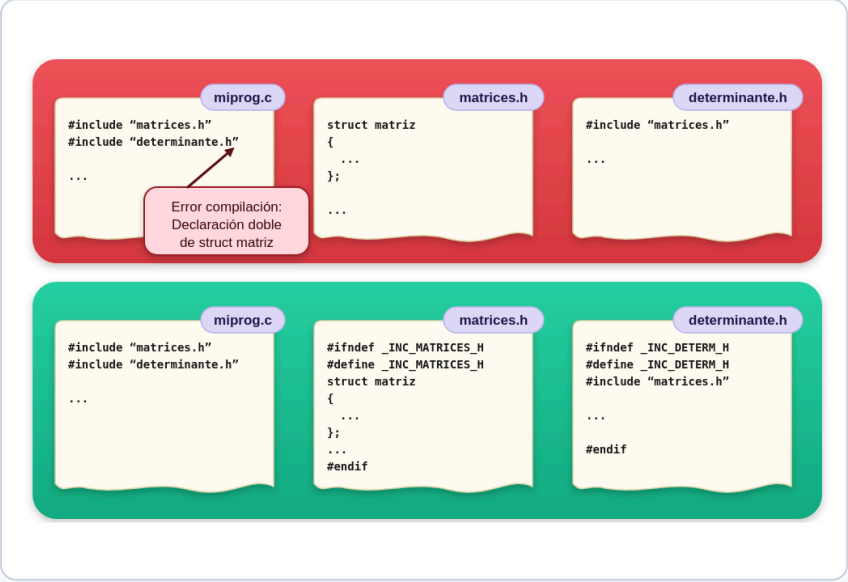
```
#define DEPURANDO 0            // cambiar por 1 ó 2

...
#if DEPURANDO > 0
    printf("Resultado parcial 1 = %d\n", ...);
#endif

#if DEPURANDO > 1
    printf("Resultado parcial 2 = %d\n", ...);
#endif
```

El preprocesador

Protección contra Inclusión Múltiple: Incluir el mismo fichero `.h` varias veces genera errores de duplicidad de tipos o estructuras en compilación. Se evita envolviendo la cabecera con la directiva `#ifndef`.



Fichero: matrices.h

```c
#ifndef _INC_MATRICES_H
#define _INC_MATRICES_H

struct matriz {
    int filas;
    int columnas;
    float* datos;
};

void InicializarMatriz(struct matriz* m);

#endif // _INC_MATRICES_H
```

Resumen del Tema

1. El preprocesador transforma textualmente el código antes de la compilación.
2. Los archivos `.c` contienen la implementación ejecutable; las cabeceras `.h` exponen las interfaces públicas.
3. Se utiliza `extern` para compartir el acceso a variables globales declaradas en otro módulo.
4. Encerrar las macros parametrizadas siempre entre paréntesis para evitar errores de evaluación.
5. Proteger todas las cabeceras `.h` mediante (`#ifndef ... #define ... #endif`).